Educational Report on the "MLB Team Wins Prediction - auto-sklearn" Notebook

Prepared from the attached Jupyter notebook, with emphasis on what each Python code cell does and how to interpret each output cell.

Subject	Beginner-friendly code walkthrough for a Moneyball analytics notebook
Target reader	Career-transition learners entering Data Science and AI
Notebook focus	Python code cells and their outputs only
Modeling task	Predicting Major League Baseball team wins

This report treats the notebook like a guided lab. For each code cell, it explains the job being performed, why that job matters in a machine learning workflow, and what the printed output means in plain language.

Table of Contents

Table of Contents 2

1. Notebook at a Glance 3

High-level workflow . . . 3

2. Code Cell 1 - Common Utilities 4

2.1 What jobs this cell performs when it runs . . . 4

2.2 Why the engineered features matter . . . 5

2.3 What the output cell means . . . 5

3. Code Cell 2 - Model 1 Configuration 6

3.1 What jobs this cell performs when it runs . . . 6

3.2 Why these settings matter . . . 6

4. Code Cell 3 - Model 1 Execution 7

4.1 What jobs this cell performs when it runs . . . 7

4.2 How to read the output step by step . . . 7

5. Code Cell 4 - Viewing the AutoML Ensemble 9

5.1 What job this cell performs . . . 9

5.2 How to interpret the output . . . 9

6. Code Cell 5 - Output File Check 10

6.1 What job this cell performs . . . 10

6.2 What the output means . . . 10

7. Overall Interpretation for a Beginner 11

7.1 Strengths visible in the notebook . . . 11

7.2 Beginner caution points . . . 11

Appendix A - Terms Used in Simple Language 12

Appendix B - Source Basis 12

1. Notebook at a Glance

The notebook is building a **regression model**, which means a model that predicts a number rather than a category. Here, the number is team wins (**W**) in a baseball season.

A useful metaphor is a professional scout with a calculator. Instead of watching only one player, the notebook watches a whole team-season record and converts raw baseball statistics into a richer team fingerprint.

High-level workflow

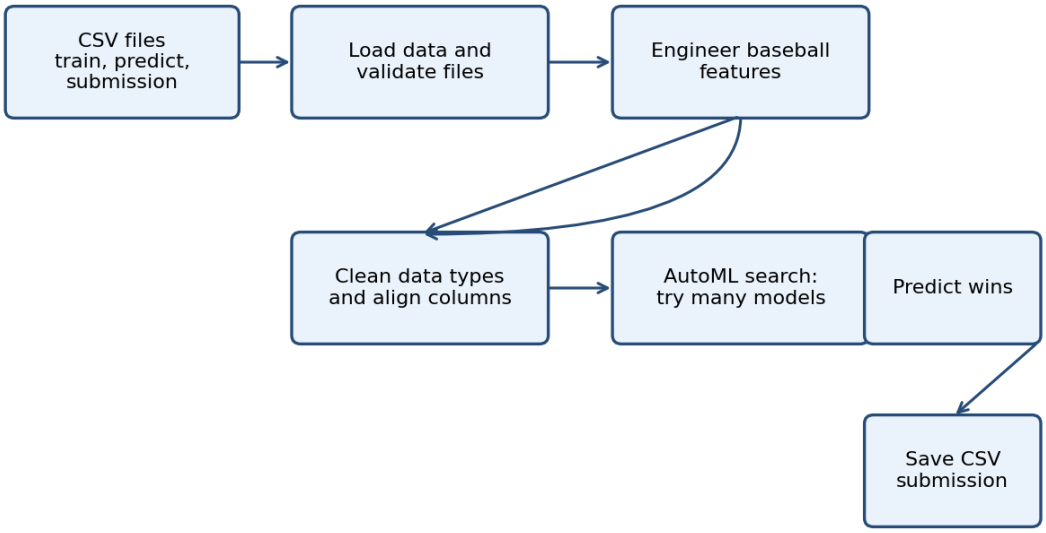

Figure 1. Simplified map of what the notebook does from raw CSV files to a final prediction file.

Code cell	Notebook role	Main job performed
Code Cell 1	Common utilities	Loads packages, sets paths, creates helper functions, engineers baseball features, cleans data, prepares training and prediction matrices.
Code Cell 2	Model 1 configuration	Stores the main auto-sklearn settings such as total search time, memory, folds, and output filename.
Code Cell 3	Model 1 execution	Loads prepared data, checks whether auto-sklearn is installed, trains the AutoML system, evaluates training error, predicts wins, and saves predictions to CSV.
Code Cell 4	Show models	Displays the set of models chosen by the auto-sklearn ensemble and their metadata.
Code Cell 5	Output check	Verifies that the expected CSV file was actually created.

2. Code Cell 1 - Common Utilities

This is the foundation cell. It does not train a model yet. Instead, it prepares the notebook workbench so that later cells can run with less repetition and fewer mistakes.

2.1 What jobs this cell performs when it runs

- **Imports the main Python tools.** Packages such as **numpy** and **pandas** handle numeric and table-like data. Several tools from scikit-learn are also imported for preprocessing and evaluation.
- **Suppresses warning noise.** The line that filters warnings tells Python not to flood the notebook with non-fatal messages.
- **Fixes the random seed.** The seed value 42 is used for both NumPy and Python's random module. This makes repeated runs more consistent.
- **Defines the project folder and input file paths.** The code changes the working folder and stores the locations of the training file, the prediction file, and the sample submission file.
- **Checks that required files exist.** The **validate_input_files()** function is a guardrail. If one file is missing, the notebook stops early with a clear error message.
- **Loads the base CSV files.** The **load_base_data()** function reads three CSV files into pandas DataFrames. A DataFrame is basically a spreadsheet-like table inside Python.
- **Creates safe division logic.** The **_safe_divide()** helper avoids divide-by-zero problems by replacing impossible divisions with missing values.
- **Engineers new baseball features.** The **add_engineered_features()** function turns raw columns into more informative measures such as OPS, WHIP, run differential, and Pythagorean winning percentage.
- **Normalizes statistics by season.** The code compares each team against the mean and standard deviation of its own season using ratio and z-score features.
- **Selects usable model features.** The **build_feature_lists()** function removes the target (**W**) and avoids leakage fields such as **WinPct_proxy**.
- **Builds a preprocessing plan.** Missing values are filled in, numbers are scaled, and categories are one-hot encoded.
- **Creates X and y.** The **get_xy()** function prepares the feature matrix **X** and the target vector **y**.
- **Cleans data types for AutoML compatibility.** Booleans are converted to integers, categorical columns can be converted to codes, object columns are coerced toward numeric values, and prediction columns are re-aligned to match training columns.
- **Prepares one reusable data bundle.** The **prepare_datasets()** function combines all prior steps and returns a dictionary containing raw data, engineered data, feature matrices, target values, grouping information, and feature lists.

2.2 Why the engineered features matter

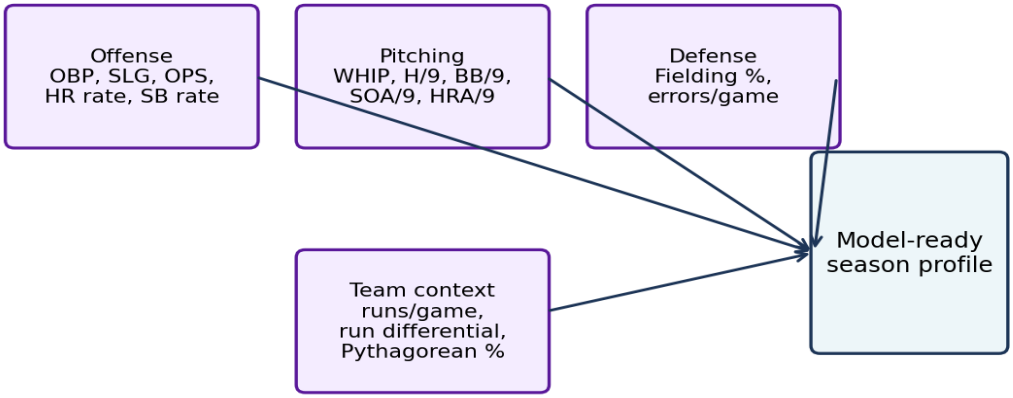

Figure 2. The notebook groups raw baseball numbers into broader performance signals that are easier for a model to use.

Feature example	What it tells the model
OPS	Combines on-base ability and slugging. In plain terms: how often a team gets on base plus how much power it hits with.
WHIP_calc	Measures how many baserunners a pitching staff allows per inning. Lower usually means tighter run prevention.
RunDiff	Runs scored minus runs allowed. This is like a season-long scoreboard balance.
PythagWinPct	An estimate of how often a team should win based on runs scored and runs allowed.
year_ratio / year_z features	Season-adjusted comparisons that tell whether a team is above or below its era average.

2.3 What the output cell means

Two stream outputs appear after this cell runs.

- **Output line 1: package warning.** The notebook prints a warning from scikit-learn about **pkg_resources** being deprecated. A warning is not the same as a crash. The cell still runs.
- **Output line 2: "Utilities loaded. Ready to run any section."** This is a manual checkpoint message written by the notebook author. It confirms that the setup code reached the end successfully.

Overall, the main meaning of this output is that the notebook environment is now ready. No predictions have been made yet.

3. Code Cell 2 - Model 1 Configuration

This cell stores **hyperparameters**. A hyperparameter is a control knob chosen by the user before training starts.

3.1 What jobs this cell performs when it runs

- **Defines the total search budget. AUTO_SKLEARN_TIME_LEFT_FOR_THIS_TASK = 7200** gives auto-sklearn up to two hours to search for good models.
- **Limits the time per candidate model. AUTO_SKLEARN_PER_RUN_TIME_LIMIT = 180** means any single candidate can use at most three minutes.
- **Sets a memory budget. 4096 MB** tells the AutoML process roughly how much RAM it can use.
- **Sets ensemble size.** The value **25** means the final solution may blend multiple model runs instead of trusting only one.
- **Defines internal cross-validation folds.** The value **5** tells auto-sklearn how many folds to use in its internal evaluation.
- **Sets parallel jobs. n_jobs = 4** allows multiple training attempts to run in parallel if the machine can support them.
- **Names temporary and output folders.** The notebook stores intermediate files and final results in named folders so runs are organized.
- **Specifies the final output filename.** The predictions should be saved as **prediction_submission_1.csv**.

3.2 Why these settings matter

A useful metaphor is a job interview process. Auto-sklearn is the hiring manager, the candidate models are job applicants, and these settings define the interview rules: how long each interview can last, how many interviews happen, and how many finalists may be kept in the final panel.

This cell has no printed output. That is normal. It quietly stores values in memory so the next cell can use them.

4. Code Cell 3 - Model 1 Execution

This is the main action cell. It is where the prepared data is handed to auto-sklearn, models are searched, a final ensemble is formed, predictions are created, and a submission file is written.

4.1 What jobs this cell performs when it runs

- Calls **prepare_datasets()**, which triggers the entire data preparation pipeline created earlier.
- Separates training and prediction objects into **X_train**, **y_train**, **X_pred**, and **submission**.
- Checks whether **auto-sklearn** is installed using a **try/except** block.
- Creates run-specific folder names by attaching a timestamp.
- Optionally removes old run folders when cleanup is enabled.
- Constructs the **AutoSklearnRegressor** with the chosen time budget, metric, ensemble size, CV folds, memory limit, and number of jobs.
- Fits the AutoML system on the training data. This is where many candidate models are tried.
- Predicts on the training data so error can be measured.
- Calculates training **MAE** (mean absolute error), which is the average size of prediction mistakes.
- Prints sprint statistics describing how many model runs happened and how many succeeded or failed.
- Predicts wins for the unseen rows in **X_pred**.
- Clips predictions into the realistic baseball range of 0 to 162 wins.
- Writes predictions into the submission template and saves the CSV file.

4.2 How to read the output step by step

- **"Using auto-sklearn tmp folder ..." and "Using auto-sklearn output folder ..."** confirm that the run-specific folders were created.
- **Repeated package warning.** The same deprecation warning appears again because the software stack is touched again during this run.
- **Warning: "No runs were available to build an ensemble from".** At those exact moments, the ensemble builder had not yet received enough completed candidate runs to combine. Because later training results appear, the search still recovered and completed useful work.
- **"Auto-sklearn training MAE: 2.5637"** means that, on the training set, the model's average miss was about **2.56 wins**.
- **Dataset name: mlb_team_wins.** A label attached to the AutoML run for bookkeeping.
- **Metric: mean_absolute_error.** Confirms the optimization target. Lower is better.
- **Best validation score: 2.698991.** This comes from internal validation during search and is more useful than the training score for judging generalization.
- **Number of target algorithm runs: 357.** Auto-sklearn tried 357 candidate training runs during the search budget.
- **Successful runs: 223.** These runs completed successfully and produced usable scores.
- **Crashed runs: 10.** A small number of crashes is not unusual in AutoML because many combinations are explored.
- **Saved predictions to prediction_submission_1.csv.** This is the final operational result of the cell.

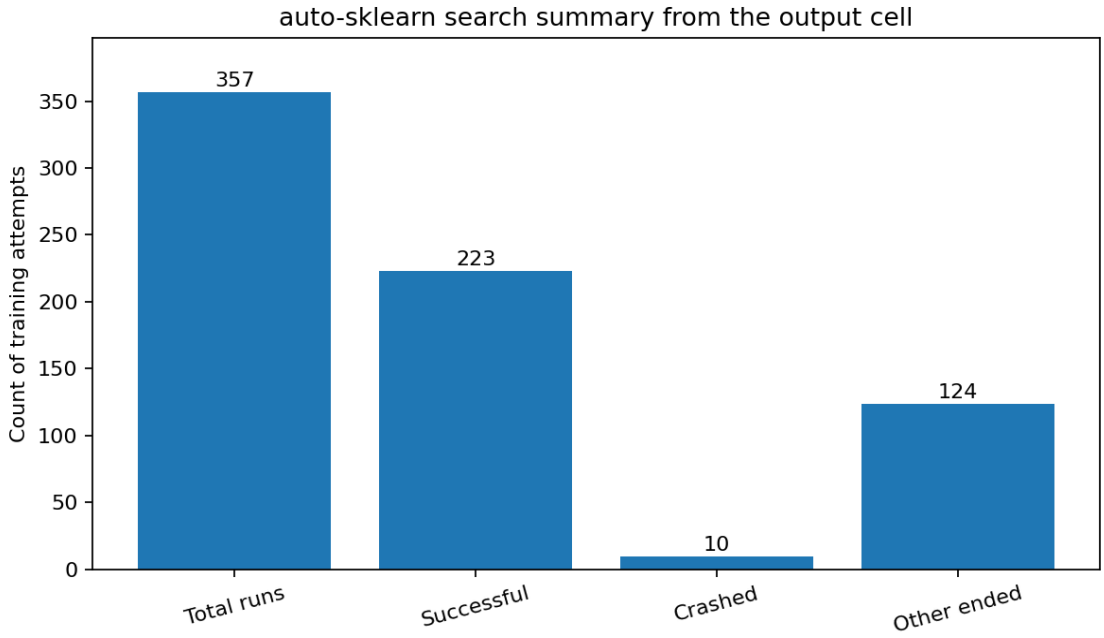

Figure 3. Beginner-friendly visual summary of the search statistics printed by auto-sklearn.

One subtle but important lesson: the notebook does not manually choose a single model family. Instead, it delegates that search to AutoML.

5. Code Cell 4 - Viewing the AutoML Ensemble

The line **automl.show_models()** asks auto-sklearn to print the models that make up the final ensemble. This is an inspection step, not a training step.

5.1 What job this cell performs

The cell reads the fitted AutoML object and prints a Python dictionary-like structure. Each top-level entry represents one model included in the final ensemble, along with metadata such as rank, cost, ensemble weight, and the underlying scikit-learn regressor.

5.2 How to interpret the output

The output is long because it exposes internals. A beginner does not need to read every parameter. The most useful concepts are **model_id**, **rank**, **ensemble_weight**, and **sklearn_regressor**.

The first visible entry in the output has rank 1 and uses **HistGradientBoostingRegressor**. Other algorithm names also appear in the full output, including **ARDRegression**, **LinearSVR**, and **AdaBoostRegressor**.

An **ensemble** works like asking several baseball analysts for their win predictions and then combining their opinions.

Algorithm name seen in output	Plain-language meaning
HistGradientBoostingRegressor	A tree-based boosting method that learns many small corrections over time. Good at capturing non-linear relationships.
ARDRegression	A linear model that automatically shrinks less useful signals. Good when many features may be noisy.
LinearSVR	A support-vector style linear regressor that looks for a stable boundary around prediction errors.
AdaBoostRegressor	A boosting method that repeatedly pays more attention to examples that were predicted poorly earlier.

For a beginner, the main meaning of this output is that auto-sklearn found several model families worth keeping and blended them into one final decision system.

6. Code Cell 5 - Output File Check

This final code cell is a lightweight verification step. It loops through a list of expected files and prints whether each one exists in the project folder.

6.1 What job this cell performs

- It creates a list containing one filename: **prediction_submission_1.csv**.
- It checks whether that file exists using **Path(f).exists()**.
- It prints either **exists** or **missing**.

6.2 What the output means

The notebook prints **prediction_submission_1.csv exists**. This confirms that the earlier model-execution cell succeeded in saving the prediction file.

A simple metaphor is checking whether the parcel was actually packed and placed at the door after preparing it.

7. Overall Interpretation for a Beginner

Seen end to end, the notebook follows a standard machine learning structure: prepare the environment, prepare the data, set the search rules, train and evaluate, inspect the solution, and save the business output.

From a Moneyball perspective, the notebook is learning from team-season summaries. It transforms box-score style statistics into a compact representation of team quality and then uses machine learning to estimate wins.

7.1 Strengths visible in the notebook

- Good modular structure through helper functions.
- Thoughtful feature engineering tied to baseball meaning.
- Era-aware normalization, which matters in historical baseball data.
- Safe guards for file checking and data alignment.
- Automatic search across multiple model families rather than a single hand-picked algorithm.

7.2 Beginner caution points

- Training MAE is informative, but validation MAE is the healthier metric for judging generalization.
- Warning suppression keeps notebooks tidy, but it can hide useful diagnostics during troubleshooting.
- Large AutoML outputs can look intimidating; focus first on the main ideas: search, evaluation metric, ensemble, and saved artifact.

Appendix A - Terms Used in Simple Language

Term	Simple meaning
DataFrame	A table in Python, similar to a spreadsheet.
Feature engineering	Creating better input variables from raw columns so the model can detect patterns more easily.
Target	The answer column the model tries to predict. Here, it is team wins (W).
Regression	A machine learning task where the output is a number.
Hyperparameter	A setting chosen before training starts, such as time budget or number of folds.
Cross-validation	A way to test a model repeatedly on different slices of the data so performance estimates are less fragile.
MAE	Mean absolute error; the average size of the prediction mistake. Lower is better.
Ensemble	A combination of multiple models whose predictions are blended.
Data leakage	Giving the model information that is too close to the answer, making evaluation look unrealistically good.
z-score	A standardized measure showing how far a value sits above or below the average in standard deviation units.

Appendix B - Source Basis

Primary source analyzed: the uploaded notebook **moneyball_analytics_autosklearn.ipynb**. All explanations in the body of the report are grounded in the visible Python code cells and their visible outputs.

This report was written for learning purposes. It explains what the notebook is doing operationally; it does not claim that the notebook is the only correct or best possible Moneyball modeling approach.